



Homework 10

Please complete the following problems. You must submit the following for each problem: a) the answer you are asked to return and b) Python code that can be run to verify your work.

1. Genome Assembly with Perfect Coverage

Although chromosomes taken from eukaryotes have a linear structure, many bacterial chromosomes are circular. You have represented a linear chromosome with a DNA string, so you only need to modify the definition of a string to model circular chromosomes.

A circular string is a string that does not have an initial or terminal element; instead, the string is viewed as a necklace of symbols. We can represent a circular string as a string enclosed in parentheses. For example, consider the circular DNA string (ACGTAC), and note that because the string "wraps around" at the end, this circular string can equally be represented by (CGTACA), (GTACAC), (TACACG), (ACACGT), and (CACGTA). The definitions of substrings and superstrings are easy to generalize to the case of circular strings (keeping in mind that substrings are allowed to wrap around).

Perfect coverage is the phenomenon in fragment assembly of having a read (or k-mer) begin at every possible location in the genome. Unfortunately, perfect coverage is still difficult to achieve, but fragment assembly technology continues to improve by leaps and bounds, and perfect coverage is perhaps not the fantasy it once was.

PROBLEM

Given: A file containing a collection of (error-free) DNA k-mers ($k \leq 50$) taken from the same strand of a circular chromosome. In this dataset, all k-mers from this strand of the chromosome are present, and their de Bruijn graph consists of exactly one simple cycle. (The assumption made above that all reads derive from the same strand is practically unrealistic; in reality, researchers will not know the strand of DNA from which a given read has been sequenced.)

Return: A cyclic superstring of minimal length containing the reads (thus corresponding to a candidate cyclic chromosome).

Sample Dataset

```
ATTAC
TACAG
GATTA
ACAGA
CAGAT
TTACA
AGATT
```

Sample Output

```
GATTACA
```



Homework

Bioinformatics Engineering

ENBC455

Hint: As always, there are many possible solutions. For one, you can build a de Bruijn graph like done in the “DBRU: Constructing a De Bruijn Graph” problem, and then create the superstring by appending the series of overlapping reads. (Make sure and check the ends of your superstring for repeating sequences that need to be trimmed.) But you may also take advantage of the problem constraints of perfect coverage and create the superstring base by base in a single direction.



2. Calling Small, Germline Variants on the Command Line

While there are a ton of variant callers on the Galaxy public servers, there may be useful tools that are not available. If you want to use these algorithms, then you have a few options. One, you can create a local Galaxy instance and install different tools, in effect customizing your local instance. There are a ton of tools available for installation in the [Galaxy Tool Shed](#) that are not installed on the public servers.

Two, you can simply download the desired program and run it on the command line. As emphasized throughout the course, command line programs are often better performing and more fully featured than their counterparts run on public servers.

Deep learning-based variant callers are becoming the standard and have achieved superior SNP calling performance using long reads. Clair3 is a small, germline variant caller for long-reads based on a deep learning algorithm. It is not on the public Galaxy servers but is available in the Galaxy Tool Shed.

PROBLEM

The easiest way to run the Clair3 program is via a Docker pre-built image that allows you to run Clair3 using a single command; see the Clair3 [README page](#) for more details. Docker is a tool that is used to automate the deployment of applications in lightweight containers so that applications can work efficiently in different environments in isolation. To install Docker Desktop (free for personal use), go [here](#).

Given: The following files, all which must be imported into your Docker container before running DeepVariant:

1. Reference genome sequence in FASTA format (import into Docker container).
2. The corresponding index file for the FASTA file (import into Docker container).
3. Aligned sequencing reads in BAM format (import into Docker container).
4. The corresponding index file for the BAM file (import into Docker container).
5. The file **platform.txt** identifying the sequencing platform used to generate the sequencing data (Oxford Nanopore sequencing or PacBio HiFi sequencing).

Return: The merge_output.vcf file generated by the Clair3 program.

Note: instead of code, please submit a screenshot of the command line after the Clair3 analysis is completed.

Hint: Docker is a popular solution for running command line programs. However, there are alternative run options listed on the Clair3 [README page](#). Below is a step-by-step guide to running Clair3 via Docker Desktop on a Windows machine. My favorite Docker tutorial is [here](#), but you can also check guides and manuals produced by Docker Inc. [here](#). I am well-aware that the Biostar Handbook loathes Docker, even stating that “the more time you spend with Docker, the worse off you’ll be.” Nonetheless, Clair3 recommends Docker, as do other bioinformatics programs, and my goal is to give you experience with different strategies to solve bioinformatics problems. You are welcome to avoid Docker usage if you wish. Again, there are alternative installation options for Clair3.



1. Download and install [Docker Desktop](#). Docker usage is free for personal and academic use.
2. Open a command line terminal. Run the **docker** command to check that everything installed correctly.
3. Find Clair3 in the [Docker Hub](#). Copy the Docker Pull Command. Paste and run the copied command on the command line. This should download the Clair3 image, which will show up in the Images section of Docker Desktop. The image is large (nearly 4 Gb), and it will be named something different like “hkubal/Clair3.”
4. To use Clair3, you must run the Clair3 image in a Docker container. The **docker run** command does this. However, **docker run hkubal/clair3** will only load up the image in a container and then shut down the container, which is not helpful for us. If we give it a command, then Docker will load the image in a container, run the given command, and shut down the container. We want to open the container and keep it running, so we can give multiple commands to Clair3 over time. We do this by adding the **-it** flag with the command **docker run -it hkubal/clair3**. This command should in essence send your terminal into a perpetually running container. If successful, you will see a location change on the command line. The default location for the Clair3 image is the **/opt/bin** folder. You can see the file directory structure for the container in Docker Desktop, just click on the Container section, select the correct container, and navigate to the Files tab.
5. Now with the container open, we need to create an input file folder and an output file folder. You can do this with the **mkdir -p "folder_name"** command. When you do, you will see the new folders pop up on the Files tab of the container in Docker Desktop.
6. The easiest way to move the input files into the correct location in the Docker container is to navigate to the newly created input file folder in Docker Desktop. You simply drag-and-drop the input files into the desired folder in Docker Desktop. But it is possible to do this task via the command line. You should import four input files (the reference FASTA, the FASTA index, the data BAM, and the BAM index).
7. Now, we can simply run Clair3 with a single, albeit long, command with many required options. The options set the input BAM file, the input reference FASTA file, the number of threads for the algorithm to use to perform the analysis (roughly equal to the number of cores in the processor of your computer, but the analysis that you will run is short and easy and it will not take long to run even if you use a small number of threads), the sequencing platform used to create the BAM file (**'ont'** for Oxford Nanopore data, **'hifi'** for PacBio data), the model used based on the sequencing platform chosen, and the output directory. Importantly, you must provide the absolute path to the input files, model, and output folder (correct: **"/opt/bin/input_folder/input_file"**; incorrect: **"input_folder/input_file"**).

- a. Command for Oxford Nanopore data:

```
run_clair3.sh --bam_fn="/opt/bin/input_folder_name/input_file_name.bam"
--ref_fn="/opt/bin/input_folder_name/input_file_name.fa" --threads=2 --platform='ont'
--model_path="/opt/models/ont" --output="/opt/bin/output_folder_name"
```

- b. Command for PacBio data:

```
run_clair3.sh --bam_fn="/opt/bin/input_folder_name/input_file_name.bam"
--ref_fn="/opt/bin/input_folder_name/input_file_name.fa" --threads=2 --platform='hifi'
--model_path="/opt/models/hifi" --output="/opt/bin/output_folder_name"
```



8. Clair3 should start running and finding small, germline variants in the data BAM file as compared to the reference FASTA file. When it is complete, multiple output files will be generated in your selected output folder. Again, the easiest way to move files between your computer and the container is to use Docker Desktop. Simply find the file in the file directory of the container in Docker Desktop, right-click the file, and select "Save." This will save the output file to your computer. The problem asks you to submit the merge_output.vcf file. (Note: if you use the small output folder for multiple runs, the previous output files will be overwritten.)
9. If you exit the container and it is still running, then you can use the `docker exec -it container_name bash` command to get container command line access back. If you exited and the container is not running, then you must first restart it with the `docker restart container_name` command.



3. Comparing Variant Callers

DeepVariant is another machine-learning based variant caller. It takes a convolutional neural network approach to converting aligned reads in a BAM file into a list of variants in a standard VCF file. Like Clair3, DeepVariant has a GitHub [page](#) and a Docker [image](#). DeepVariant is also installed on public Galaxy servers, offering an easier way to run the program as compared to Clair3.

Most variant calling algorithms will identify different sets of variants from the same input/BAM file. Thus, there is always value in running multiple variant callers on the same file and comparing their results. If the different algorithms do identify different variants, then it is straightforward to combine the different VCF files into a single, comprehensive file containing all identified variants.

The “bcftools stats” program is available on public Galaxy servers and is a simple, easy method to assess the makeup of VCF files. This tool parses such files and produces stats that can be plotted. When two VCF files are entered, the program also produces stats on the intersections and complements of the two input files, counting the number of variants that are in both VCF files or in only one file.

PROBLEM

Given: The following files, three of which must be uploaded into Galaxy before running the DeepVariant and bcftools stats programs:

1. Reference genome sequence in FASTA format (upload to Galaxy; the same FASTA file as in the “Calling Small, Germline Variants on the Command Line” problem).
2. Aligned sequencing reads file in BAM format (upload to Galaxy; the same BAM file as in the “Calling Small, Germline Variants on the Command Line” problem).
3. VCF file produced by the Clair3 algorithm in the “Calling Small, Germline Variants on the Command Line” problem (named “merge_output.vcf” by Clair3).
4. The file **platform.txt** identifying the sequencing platform used to generate the sequencing data (Oxford Nanopore sequencing or PacBio HiFi sequencing). This will determine the “Model type” setting for the DeepVariant tool.

Return: The percentage of SNPs called by the DeepVariant algorithm that were also called by the Clair3 algorithm (data that can be found in the PDF output of the bcftools stats program).

Note: instead of code, please submit a screenshot of your Galaxy history showing successful runs of the DeepVariant and bcftools stats programs.

Hint: It is easy to run DeepVariant on the Galaxy public servers. The input files are a reference genome in FASTA format and the aligned reads file in BAM format. The only other setting to change is the “Model type: setting, which corresponds to the sequencing platform that generated the reads/data. Note that DeepVariant is a rather slow algorithm, even when using relatively small input files. **Do not start the program too close to the assignment deadline, or it may not finish in time.**

For bcftools stats, you need to enter the two VCF files to be compared into the “VCF/BCF Data *” and “Optional VCF/BCF Data to compare against” boxes. There are two options that you should change from default. Under the “Stats” menu, select “Use a genome from history” for the “Choose reference genome” setting and make sure that



Homework

Bioinformatics Engineering

ENBC455

the given FASTA sequence is selected. Finally, for the “Create a plots pdf with this title” setting, enter a descriptive title of some sort to prompt the program to generate useful tables and plots in PDF format.

The information requested by the problem can be found in the first data table in the PDF file generated by the bcftools stats program. The “n” column under the “SNPs” heading reports the number of SNPs identified only in VCF file #1, only in VCF file #2, and in both VCF files.